

Q.No:9

Question description

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n . You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Constraints

$2 \leq n \leq 500$
 $1 \leq m \leq 1000$
 $1 \leq a, b \leq n$

Input

The first input line has two integers n and m : the number of rooms and teleporters. The rooms are numbered $1, 2, \dots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b : there is a teleporter from room a to room b . There are no two teleporters whose starting and ending room are the same.

Output

First print an integer k : the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

▼ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) <pre>6 7 1 2 1 3 2 6 3 4 3 5 4 6 5 6</pre> EXPECTED OUTPUT <pre>2 4 1 3 5 6 3 1 2 6</pre>	INPUT (STDIN) <pre>6 7 1 4 3 5 4 6 5 6 2 2 1 3 2 6</pre> EXPECTED OUTPUT <pre>2 4 1 3 5 6 3 1 4 6</pre>

▼ Mandatory Test Cases

Test Case 1
KEYWORD <pre>int bfs(int n,int s,int t)</pre>

Code:

```
#include <stdio.h>
#include <string.h>

#define MAXN 505
#define MAXE 5005
#define INF 1000000000

int head[MAXN];
int to[MAXE];
int next[MAXE];
int cap[MAXE];
```

```
int edgeCount = 0;

int parentEdge[MAXN];

void link(int u, int v)
{
    to[edgeCount] = v;
    cap[edgeCount] = 1;
    next[edgeCount] = head[u];
    head[u] = edgeCount++;

    to[edgeCount] = u;
    cap[edgeCount] = 0;
    next[edgeCount] = head[v];
    head[v] = edgeCount++;
}

int bfs(int n, int s, int t)
{
    int queue[MAXN];
    int front = 0;
    int rear = 0;

    int visited[MAXN];
    memset(visited, 0, sizeof(visited));

    queue[rear++] = s;
    visited[s] = 1;
    parentEdge[s] = -1;

    while (front < rear)
    {
        int u = queue[front++];
        int e;

        for (e = head[u]; e != -1; e = next[e])
        {
            int v = to[e];

            if (!visited[v] && cap[e] > 0)
            {
                visited[v] = 1;
                parentEdge[v] = e;
                queue[rear++] = v;

                if (v == t)
                    return 1;
            }
        }
    }

    return 0;
}
```

```
}

int used[MAXN];
int route[MAXN];
int routeSize;

int findPath(int u, int t)
{
    int e;

    if (u == t)
        return 1;

    used[u] = 1;

    for (e = head[u]; e != -1; e = next[e])
    {
        int v = to[e];
        if (e % 2 == 0 && cap[e + 1] > 0 && !used[v])
        {
            cap[e + 1]--;
            route[routeSize++] = v;

            if (findPath(v, t))
                return 1;

            routeSize--;
            cap[e + 1]++;
        }
    }

    used[u] = 0;
    return 0;
}

int main()
{
    int n, m;
    int i;
    int a, b;
    int maxFlow = 0;

    scanf("%d %d", &n, &m);

    for (i = 1; i <= n; i++)
        head[i] = -1;

    for (i = 0; i < m; i++)
    {
        scanf("%d %d", &a, &b);
        link(a, b);
    }
}
```

```
while (bfs(n, 1, n))
{
    int v = n;

    while (v != 1)
    {
        int e = parentEdge[v];

        cap[e]--;
        cap[e ^ 1]++;

        v = to[e ^ 1];
    }

    maxFlow++;
}

printf("%d\n", maxFlow);

for (i = 0; i < maxFlow; i++)
{
    routeSize = 1;
    route[0] = 1;

    memset(used, 0, sizeof(used));

    findPath(1, n);

    printf("%d\n", routeSize);

    int j;
    for (j = 0; j < routeSize; j++)
    {
        if (j > 0)
            printf(" ");

        printf("%d", route[j]);
    }

    printf("\n");
}

return 0;
}
```

Output:

Test_case 1:

```
● aadityajaiswal@Aad...  
adityajaiswal/Desktop  
6 7  
1 2  
1 3  
2 6  
3 4  
3 5  
4 6  
5 6  
2  
4  
1 3 5 6  
3  
1 2 6  
ⓧ aadityajaiswal@Aad...
```

Test_case2:

```
adityajaiswal/Desktop  
6 7  
1 4  
3 5  
4 6  
5 6  
2 2  
1 3  
2 6  
2  
4  
1 3 5 6  
3  
1 4 6  
: aadityajaiswal@Aad...
```